DSA4213: Assignment 1

Language Models, Embeddings, and Representations

Natural Language Processing for Data Science

Doudou Zhou Semester 1, AY2026/2027

Last updated on: **2026/09/08.**

Overview

This is an **individual assignment**. It covers language modeling, word embeddings, RNNs, LSTMs, Transformers, and learned representations. You will compare model families on a common corpus, examine the effect of embedding choices, and evaluate whether the learned representations help a downstream task. Part III reuses a trained language model from Part I or II and may use a different dataset.

This assignment is worth **30%** of the course grade.

Due Date

- **Due**: October 16, 2026, at 11:59 pm (Singapore time).
- Late submission policy and submission platform will be announced separately.

General Requirements

- Choose your own English text corpus for Parts I and II. For Part III, choose a suitable labeled dataset for your downstream task and use representations from a language model trained in Part I or II.
- Train the required language models yourself from scratch. Part II explicitly allows public pretrained word embeddings. You may use PyTorch, TensorFlow, HuggingFace, `gensim`, and other standard libraries, and adapt the course notebooks with attribution.
- Choose your own dataset size and model size, keeping the scale computationally reasonable. You may use a manageable subset of the data.
- Comparisons should be reasonable and clearly explained.

Part I: Language Model Training and Comparison (40 points)

Using the same training corpus, train and compare the following next-token language models:

- n-gram language model with smoothing or interpolation;
- RNN language model;
- LSTM language model;

- Transformer language model with causal self-attention.

Use randomly initialized, trainable input embeddings for the three neural models. Select architectures, hyperparameters, context lengths, and decoding settings as appropriate, and document your choices. All models must predict each target using only preceding tokens.

Your comparison should include:

- **Prediction quality**: report test perplexity or average next-token cross-entropy on the common evaluation setup. Exclude padding tokens from the loss.
- **Training and inference cost**: report training time, model size, and inference time or throughput. State the hardware, training budget, and timing procedure; use a common inference workload.
- **Learning behavior**: show training and validation loss curves for the neural models and discuss convergence or overfitting.
- **Generated text**: compare continuations from the same small set of prompts using matched decoding settings and generation lengths. Discuss concrete differences such as repetition, local fluency, and consistency.

Explain the main tradeoffs supported by your results. Separate observed differences from possible explanations involving architecture, parameter count, context length, or training budget.

Part II: Embedding Variants and Ablation (30 points)

Select **one** neural architecture from Part I and compare these three input-embedding settings:

- randomly initialized embeddings learned jointly with the language model;
- embeddings trained by yourself on the training corpus using **one** method, such as Skip-gram, CBOW, or GloVe, and then kept fixed during language-model training;
- public pretrained word embeddings, such as GloVe, kept fixed during language-model training.

Keep the embedding dimension, remaining architecture, data, and training procedure the same across the three settings. Choose the dimension with the public embeddings in mind; document vocabulary coverage and how missing words are initialized. In the fixed settings, keep the entire input-embedding table fixed.

Compare test prediction quality, convergence, training stability, and representative generated text. Report the cost of training your own embeddings separately from language-model training. Interpret the comparison in light of both embedding trainability and the additional training data used by public embeddings.

For the first setting, reuse your Part I model if its embedding dimension, architecture, data, and training procedure match this comparison. Then train **two new language models**: one with your own frozen embeddings and one with public frozen embeddings. Train their remaining parameters from scratch. These are the two additional language-model runs; training your own word embeddings is a separate step. Testing every embedding setting on every architecture is optional.

Part III: Downstream Representation Analysis (20 points)

Use the models trained in Parts I–II for a simple supervised task, such as sentiment or topic classification. Compare **at least two representation choices** on the same labeled data. For example, compare representations from an LSTM and a Transformer, from two embedding variants, or from two pooling methods applied to one trained model.

Describe how each input becomes a fixed-length representation, such as a final hidden state or an average of token states excluding padding. Use the same simple classifier family and evaluation procedure for both choices. Freezing the language models and training only the classifiers is sufficient; fine-tuning is optional and should be applied consistently across the comparison.

Report a suitable held-out task metric, such as accuracy or macro-F1, and examine representative errors. Explain which representations work better in your experiment. If you compare different language models, discuss whether their language-model performance ranking agrees with their downstream ranking.

Experimental Focus

Organize your discussion around a clear comparison question, such as how model families differ on your corpus, whether fixed embeddings help, or whether better next-token prediction leads to better classification. The required experiments may be sufficient to answer your question; additional models or large hyperparameter searches are not required.

Submission

Submit a PDF report and your code, either as a GitHub link accessible to the instructor or a zip file. The report should be at most **6 pages**, excluding appendices and references. Keep the main results and analysis in the main report.

The report should focus on results and comparisons; unnecessary background will not be rewarded. The report should clearly state:

- the dataset used and preprocessing decisions;
- the models and variants compared;
- the evaluation criteria;
- the main quantitative and qualitative observations;
- the most important limitations, failure cases, and lessons learned.

Your code should include a `README.md` with data access and preprocessing instructions, dependencies, commands, configurations, and random seeds needed to reproduce the main results. Do not submit API keys, private credentials, or data that you do not have permission to share.

Grading Guidelines

Marks reflect correct implementation, fair comparisons, supporting evidence, and clear interpretation. Strong submissions explain specific successes and failures, connect results across the three

parts, and distinguish observations from hypotheses. Missing required comparisons, data leakage, unsupported conclusions, and missing reproducibility information will reduce marks. A larger model or a better raw score alone does not earn higher marks.

Rubric

Component	**Points**
Part I: language model implementation, comparison, and interpretation	40
Part II: embedding variants, ablation design, and analysis	30
Part III: downstream task and representation analysis	20
Report clarity, reproducibility, and responsible tool/data use	10
Total	**100**

Academic Integrity and Tool Use

You may use standard libraries, tutorials, documentation, pretrained embeddings, LLMs, and coding assistants. You must clearly cite external code, datasets, pretrained models, and substantial assistance from tools. Grades will be based on your problem formulation, implementation, experimental evidence, evaluation quality, interpretation, reproducibility, and ability to explain the results. The submitted analysis and interpretation should reflect your own understanding.